САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа 5: изучение основ работы с менеджером зависимостей npm

Выполнил:

Якшин Артемий

K3440

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Изучить основные команды пакетного менеджера npm и научиться стартовать проект на Vue. Работа выполняется по мануалу курса «Небольшой мануал по работе с Vue.JS». В качестве отчёта нужны скриншоты компонентов и код получившегося приложения.

По мануалу собрано приложение «Заметки» на Vue 3 с Vue Router, Pinia, axios и Bootstrap. Код лежит в папке `hw5/notes-app`.

Ход работы

1. Node.js и npm

npm устанавливается вместе с Node.js. Версии проверяются командами:

```
$ node -v
v25.2.1
$ npm -v
11.6.2
```

В мануале упоминаются LTS-версии 20.17 и 10.8. Текущий генератор проектов create-vue требует Node.js не ниже 22.18 (поле `"engines"` в созданном `package.json`), поэтому использована установленная версия 25.

2. Создание проекта

Мануал предлагает сначала глобально установить `@vue/cli`, а затем выполнить `npm init vue@latest`. Для второй команды Vue CLI не нужен: `npm init vue@latest` (то же, что `npm create vue@latest`) сам скачивает пакет `create-vue` и запускает его, как `npx`. Vue CLI сейчас находится в режиме поддержки, поэтому глобальная установка пропущена. Ответы на вопросы мастера переданы флагами: Vue Router, Pinia и ESLint, без TypeScript, JSX и тестов.

```
$ npm create vue@latest notes-app -- --router --pinia --eslint
Scaffolding project in …/hw5/notes-app...
└  Done. Now run:
   cd notes-app
   npm install
   npm run dev
```

Затем установлены зависимости и в `package.json` добавлены скрипты из мануала: `start` запускает Vite на порту 8080, а `api` запускает json-server с заметками.

```
"scripts": {
  "start": "vite --port 8080",
  "dev": "vite",
  "build": "vite build",
```

```
  "preview": "vite preview",
  "api": "json-server --watch db.json --port 3000",
  "lint": "run-s \"lint:*\"",
  "lint:oxlint": "oxlint . --fix",
  "lint:eslint": "eslint . --fix --cache"
}
```

```
$ npm start
> notes-app@0.0.0 start
> vite --port 8080

  VITE v8.3.7  ready in 413 ms
  ➜  Local:   http://localhost:8080/
  ➜  Vue DevTools: Press Option(⌥)+Shift(⇧)+D in App to toggle the Vue DevTools
```

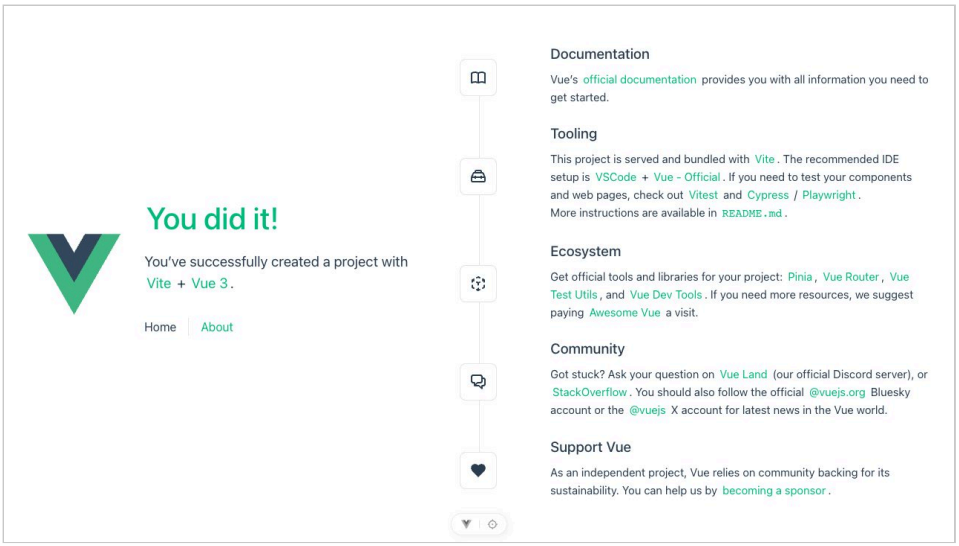


Рисунок 1. Стартовая страница нового проекта по адресу localhost:8080

3. Основные команды npm

В ходе работы использованы и разобраны команды npm, которые нужны для повседневной работы с проектом:

Таблица 1. Команды npm

Команда	Что делает	Пример из проекта
`npm init` / `npm create`	создаёт `package.json` или запускает генератор проекта `create-*`	`npm create vue@latest notes-app`
`npm install`	ставит всё из `package.json` в `node_modules`, создаёт или обновляет `package-lock.json`	added 251 packages in 32s
`npm i <пакет> -S`	ставит пакет и записывает его в `dependencies` (с npm 5 флаг `-S` действует по умолчанию)	`npm i axios pinia-persists bootstrap -S`
`npm i -D <пакет>`	ставит пакет в `devDependencies`: он нужен только при разработке	`npm i -D json-server@0.17.4`
`npm run <скрипт>`	запускает скрипт из раздела `scripts`; без имени показывает список скриптов	`npm run api`, `npm run build`
`npm start`	сокращение для `npm run start`	Vite на порту 8080
`npm ls --depth=0`	дерево установленных пакетов верхнего уровня	axios@1.21.1, bootstrap@5.3.8, pinia@4.1.0, vue@3.5.45…
`npm view <пакет>`	информация о пакете из реестра без установки	`npm view axios version license` → 1.21.1, MIT
`npm outdated`	какие пакеты можно обновить и до какой версии	json-server 0.17.4, последняя 1.0.0-beta.15
`npm audit`	поиск известных уязвимостей в зависимостях	4 moderate в dev-зависимостях, 0 с `--omit=dev`
`npm uninstall <пакет>`	удаляет пакет из `node_modules` и `package.json`	—
`npm ci`	чистая установка строго по `package-lock.json` (для CI и проверки проекта)	использовалась для повторной установки
`npx <команда>`	запускает исполняемый файл пакета без глобальной установки	`npx create-vue --help`

Номера версий в `package.json` записаны по правилам semver: `"axios": "^1.21.1"` разрешает обновления до любой версии 1.x.x не ниже 1.21.1. Точные версии, которые реально установлены, фиксирует `package-lock.json`, поэтому у всех, кто выполнит `npm ci`, будет одинаковый набор пакетов. Папка `node_modules` в репозиторий не попадает (она есть в `.gitignore`), её всегда можно восстановить командой `npm install`.

```
$ npm outdated
Package              Current   Wanted        Latest  Location
```

```
eslint-plugin-oxlint   1.82.0   1.82.0         1.93.0  node_modules/eslint-plugin-
oxlint
json-server            0.17.4   0.17.4  1.0.0-beta.15  node_modules/json-server
oxlint                 1.82.0   1.82.0         1.93.0  node_modules/oxlint
...
```

json-server намеренно закреплён на версии 0.17: в бета-версии 1.0 изменилась работа с параметрами запросов, а курс использует именно 0.17.

4. axios, Pinia и Bootstrap

Как в мануале, создан один общий экземпляр axios и класс `NotesApi`, который получает этот экземпляр в конструкторе (внедрение зависимости). Файл `src/api/index.js` собирает модули API в одном месте:

```
// src/api/instance.js
import axios from 'axios'

const apiURL = 'http://localhost:3000'
const instance = axios.create({ baseURL: apiURL })

export default instance

// src/api/notes.js
class NotesApi {
  constructor(instance) {
    this.API = instance
  }

  getAll = async () => this.API({ url: '/notes' })

  createNote = async (data) => this.API({
    method: 'POST',
    url: '/notes',
    data,
    headers: { 'Content-Type': 'application/json' }
  })

  deleteNote = async (id) => this.API({ method: 'DELETE', url: `/notes/${id}` })
}

// src/api/index.js
const notesApi = new NotesApi(instance)
export { notesApi }
```

Pinia подключена с плагином `pinia-persists`: состояние хранилища сохраняется в `localStorage` под ключом `pinia_notes` и восстанавливается после перезагрузки. Пакет довольно старый (2022 год), но устроен просто, через `watch` за `$state`, и с Pinia 4 работает без изменений.

```
// src/stores/index.js
import { persist } from 'pinia-persists'
import { createPinia } from 'pinia'
```

```
const pinia = createPinia()
pinia.use(persist())

export default pinia

// src/stores/notes.js
const useNotesStore = defineStore('notes', {
  state: () => ({ notes: [] }),

  actions: {
    async loadNotes() {
      const response = await notesApi.getAll()
      this.notes = response.data
      return response
    },

    async createNote(data) {
      const response = await notesApi.createNote(data)
      // в мануале: this.notes = response.data — но сервер возвращает одну заметку
      this.notes.push(response.data)
      return response
    },
    ...
  }
})
```

В `main.js` подключены хранилище, роутер, стили и скрипты Bootstrap, а `App.vue` содержит только `<router-view />`:

```
import { createApp } from 'vue'
import App from '@/App.vue'
import router from '@/router'
import store from '@/stores'

import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap'
import '@/assets/main.css'

const app = createApp(App)
app.use(store)
app.use(router)
app.mount('#app')
```

5. Компоненты и роутинг

По мануалу код разделён на три вида компонентов:

- **представление** `views/NotesPage.vue` — страница, на которую ведёт маршрут;
- **компонент** `components/NoteCard.vue` — отдельная часть страницы со своей логикой;
- **лейаут** `layouts/BaseLayout.vue` — обёртка, которая задаёт контейнер, а содержимое страницы получает через `<slot />`.

```
<!-- src/layouts/BaseLayout.vue -->
<template>
  <main class="container my-2">
    <slot />
  </main>
</template>

<!-- src/components/NoteCard.vue -->
<template>
  <article class="card note-card h-100">
    <div class="card-body">
      <h2 class="card-title h5">{{ name }}</h2>
      <p v-if="text" class="card-text">{{ text }}</p>
      <p v-else class="card-text text-body-secondary">Без текста</p>
    </div>
    <div class="card-footer bg-transparent border-0 pt-0">
      <button class="btn btn-sm btn-outline-danger" type="button"
              :aria-label="`Удалить заметку «${name}»`"
@click="$emit('delete')">Удалить</button>
    </div>
  </article>
</template>

<script>
export default {
  name: 'NoteCard',
  props: {
    name: { type: String, required: true },
    text: { type: String, required: false }
  },
  emits: ['delete']
}
</script>
```

К карточке из мануала добавлена кнопка «Удалить». Карточка не удаляет заметку сама, а отправляет родителю событие `delete`: данные передаются вниз через свойства, а действия поднимаются вверх через события.

Маршрут `/` загружает страницу лениво, через динамический `import()`. Поэтому при сборке код страницы попадает в отдельный файл `NotesPage-….js`.

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/', name: 'notes', component: () => import('../views/NotesPage.vue') }
  ]
})
```

6. Страница заметок: v-model, v-for, v-if, methods, mounted

Страница написана на Options API, как в мануале:

- `data()` хранит поля формы;
- `mapState` подключает к `computed` массив `notes` из хранилища;

- `mapActions` добавляет в `methods` действия хранилища;
- хук `mounted` загружает заметки, когда компонент появился на странице.

```
<form ref="noteForm" class="note-form d-flex flex-column gap-2 my-4"
@submit.prevent="createCard">
  <label class="form-label mb-0" for="noteName">Заголовок</label>
  <input id="noteName" v-model="form.name" type="text" class="form-control" required>
  <label class="form-label mb-0 mt-2" for="noteText">Текст</label>
  <textarea id="noteText" v-model="form.text" class="form-control" rows="4" />
  <button type="submit" class="btn btn-primary align-self-start mt-
2">Отправить</button>
</form>

<p v-if="!notes.length" class="text-body-secondary">Заметок пока нет — добавьте
первую.</p>
<div v-else id="notes" class="row row-cols-1 row-cols-md-2 g-4 mt-2">
  <div v-for="note in notes" :key="note.id" class="col">
    <note-card :name="note.name" :text="note.text" @delete="deleteNote(note.id)" />
  </div>
</div>
```

```
export default {
  name: 'NotesPage',
  components: { BaseLayout, NoteCard },

  data() {
    return { form: { name: '', text: '', userId: 1 } }
  },

  computed: {
    ...mapState(useNotesStore, ['notes'])
  },

  methods: {
    ...mapActions(useNotesStore, ['loadNotes', 'createNote', 'deleteNote']),

    async createCard() {
      await this.createNote(this.form)
      // в мануале: this.$refs.noteForm.reset() — очищает только поля в DOM,
      // а значения в data остаются, поэтому сбрасываем сами данные
      this.form.name = ''
      this.form.text = ''
    }
  },

  mounted() {
    this.loadNotes()
  }
}
```

В мануале нашлись два места, которые работали неправильно, и они исправлены:

- действие `createNote` записывало в `notes` ответ сервера, то есть одну созданную заметку вместо массива;

- `this.$refs.noteForm.reset()` очищал поля на экране, но не данные `v-model`. Следующая отправка ушла бы со старыми значениями, хотя поля выглядят пустыми.

Заметки хранятся в `db.json` и отдаются командой `npm run api` (json-server). Приложение загружает три заметки (рис. 2). После заполнения формы (рис. 3) и отправки новая заметка появляется в конце списка, а поля очищаются (рис. 4).

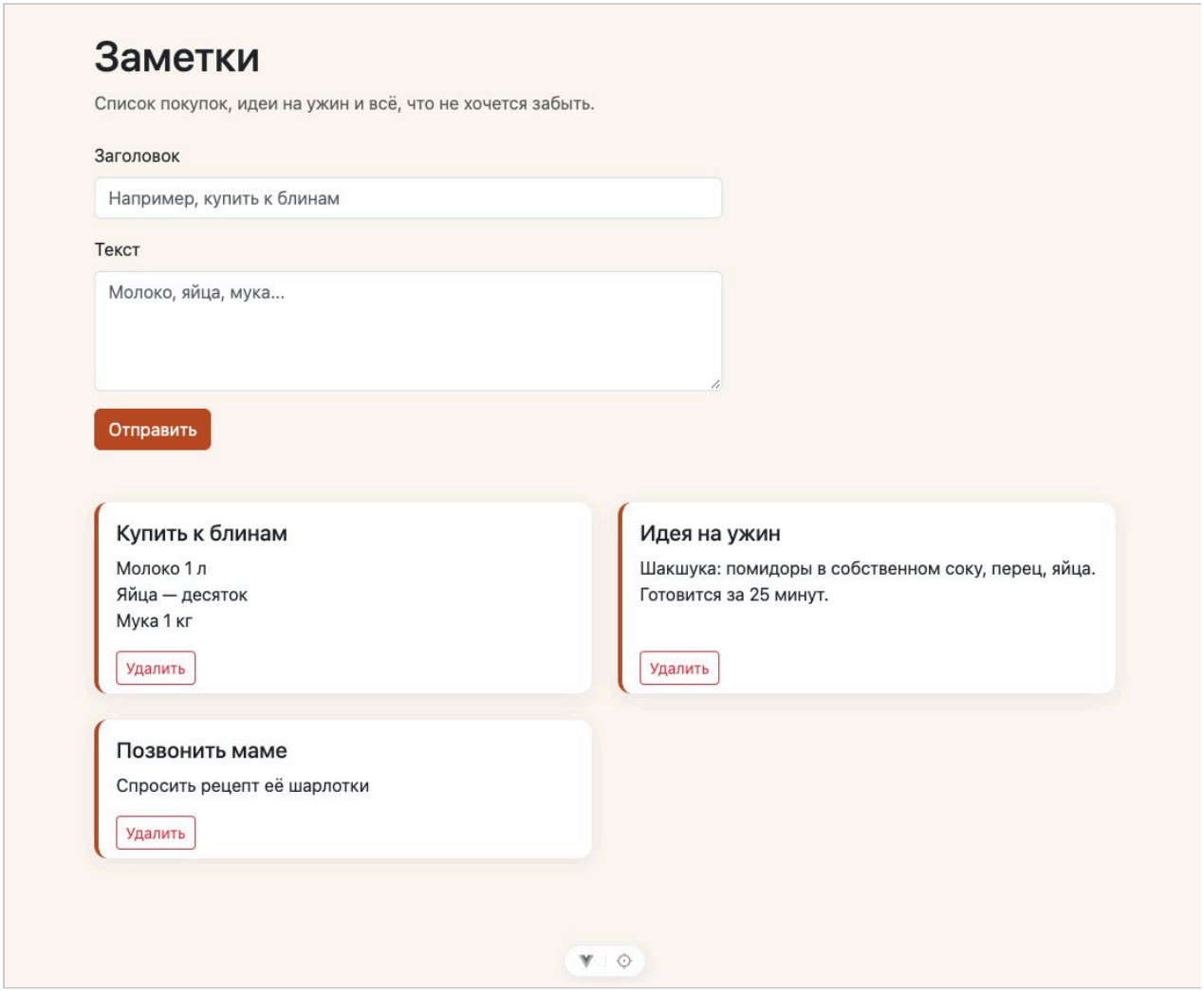


Рисунок 2. Страница заметок: данные пришли с json-server

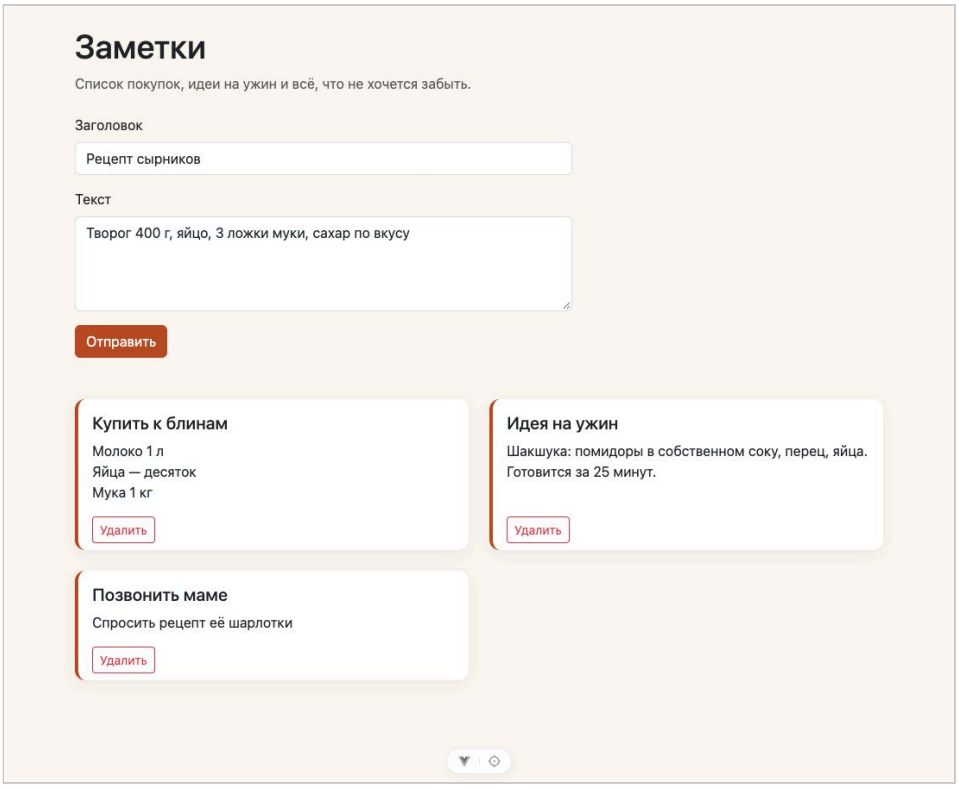


Рисунок 3. Заполненная форма (v-model)

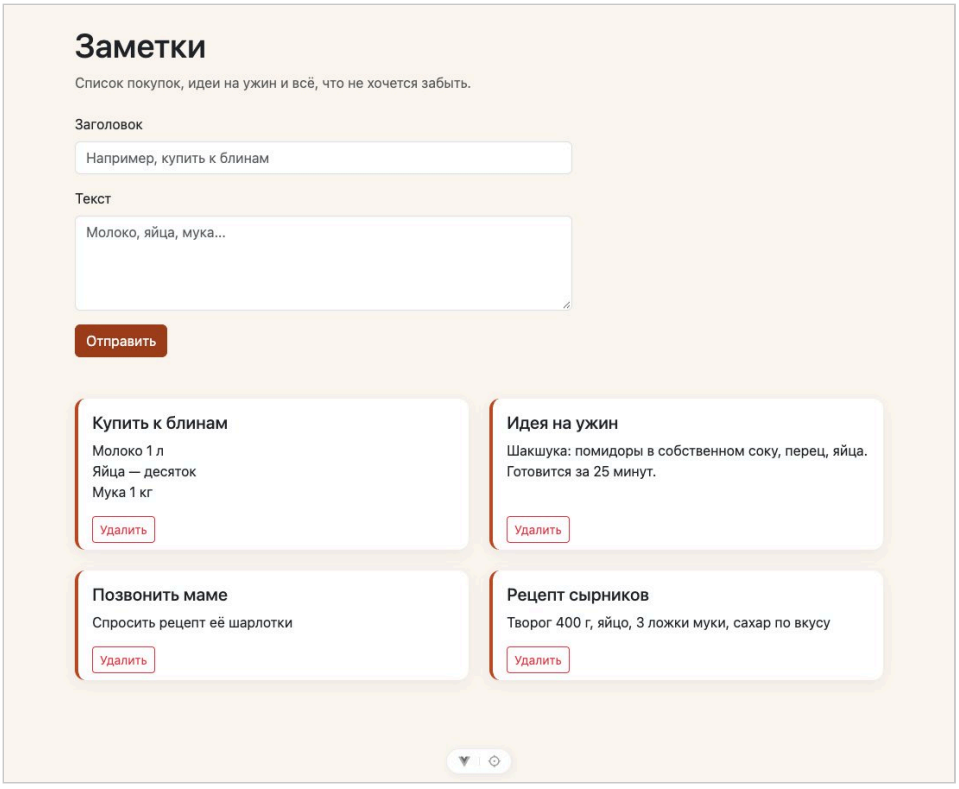


Рисунок 4. Заметка добавлена, форма очищена

7. Компоненты в Vue DevTools

create-vue подключает плагин `vite-plugin-vue-devtools`, и панель открывается прямо в приложении по Option+Shift+D. В ней видно дерево компонентов `Root → RouterView →`

`NotesPage → BaseLayout → NoteCard`. У `NotesPage` видны данные формы и вычисляемое свойство `notes` из хранилища, а также ссылка `noteForm` (рис. 5). У `NoteCard` видны свойства `name` и `text` и объявленное событие `delete` (рис. 6).

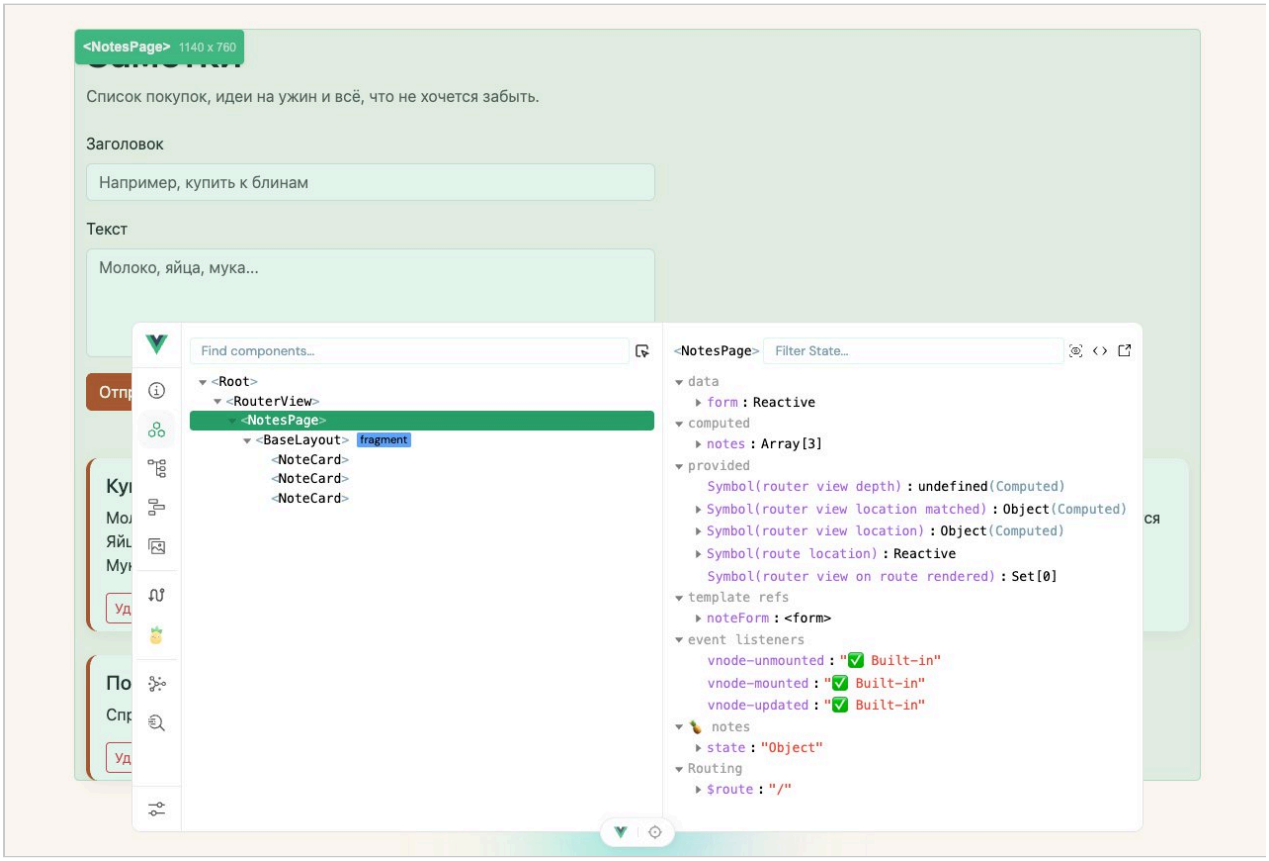


Рисунок 5. Vue DevTools: компонент NotesPage

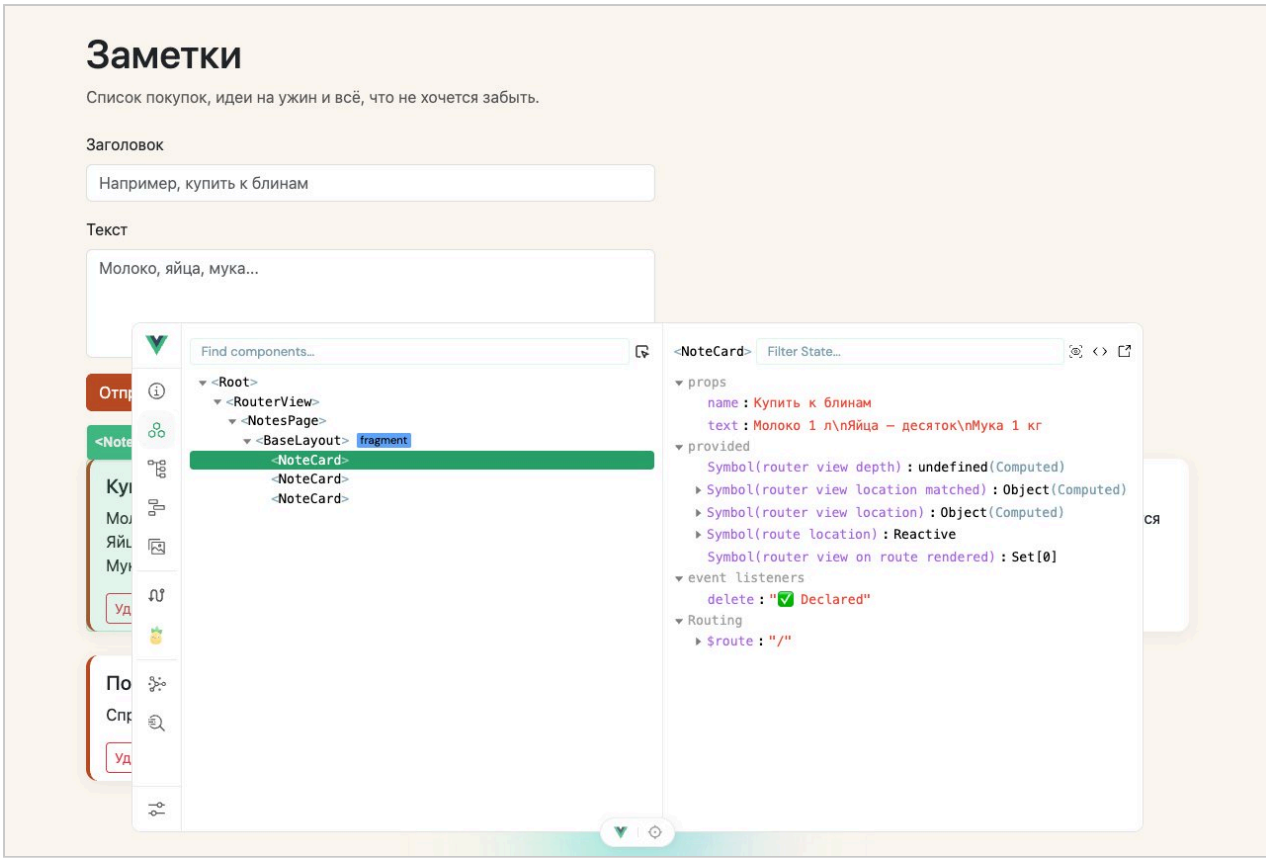


Рисунок 6. Vue DevTools: компонент NoteCard и его свойства

8. Проверка

Линтер (`npm run lint`: oxlint и ESLint) замечаний не выдал, сборка проходит:

```
$ npm run build
dist/index.html                          0.43 kB │ gzip:   0.30 kB
dist/assets/NotesPage-T8twqU5Z.css       0.18 kB │ gzip:   0.15 kB
dist/assets/index-CJkMW1-m.css         230.33 kB │ gzip:  30.82 kB
dist/assets/NotesPage-CpqHA4Y-.js       61.04 kB │ gzip:  23.15 kB
dist/assets/index-DInedCCv.js          325.18 kB │ gzip: 115.15 kB
✓ built in 147ms
```

Сценарий проверен в браузере без ошибок в консоли:

- загрузка трёх заметок;
- создание четвёртой с очисткой формы;
- сохранение состояния в `localStorage` и после перезагрузки;
- удаление заметки.

Вывод

В работе разобраны основные команды npm: создание проекта, установка зависимостей и dev-зависимостей, запуск скриптов, просмотр, обновление и аудит пакетов. Разобрана и

роль файлов `package.json` и `package-lock.json`. По мануалу с нуля собрано приложение на Vue 3 со всеми его частями:

- роутер с ленивой загрузкой страницы;
- хранилище Pinia с сохранением в `localStorage`;
- слой API на axios с внедрением зависимости;
- деление на представление, компонент и лейаут.

В приложении использованы `v-model`, `v-for`, `v-if`, свойства и события компонентов, `methods` и хук `mounted`. Попутно исправлены две ошибки из мануала. Эта основа пригодится в ЛР3 при переводе «Поварёшки» на Vue.